

@teach_ing4u



MACHINE LEARNING = (ML) =

COMPLETE
SHORT NOTES

1. INTRODUCTION TO MACHINE LEARNING

What is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that allows computers to learn from data and make predictions or decisions without being explicitly programmed.

Why ML?

- ✓ Learns from Experience
- ✓ Improves with more data
- ✓ Handles complex problems
- ✓ Automates decisions

Types of Machine Learning

① Supervised Learning

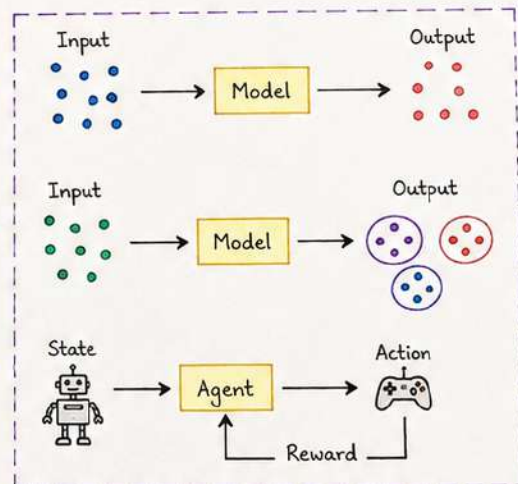
- Learn from labeled data
- Example: Email spam detection

② Unsupervised Learning

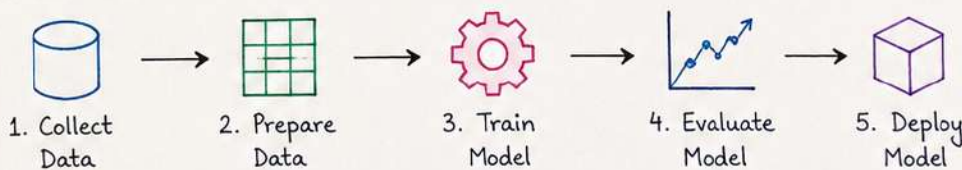
- Learn from unlabeled data
- Example: Customer segmentation

③ Reinforcement Learning

- Learn by interacting with environment
- Example: Game playing, Robotics



ML Workflow



Applications of ML

- | | | |
|--------------------------|----------------------|--------------------------|
| • Email Spam Detection | • Fraud Detection | • Medical Diagnosis |
| • Recommendation Systems | • Speech Recognition | • Predictive Maintenance |
| • Face Recognition | • Self-driving Cars | • Stock Price Prediction |

"ML is all about learning patterns from data and making smart decisions." ♥

* 2. DATA TYPES & PREPROCESSING *

Data Types

- ① Numerical Data
 - Continuous (e.g., height, weight)
 - Discrete (e.g., number of cars)
- ② Categorical Data
 - Nominal (e.g., color, gender)
 - Ordinal (e.g., low, medium, high)
- ③ Text Data
 - Reviews, comments, documents
- ④ Time Series Data
 - Sales over time, stock prices, etc.

Why Preprocess Data?

- ✓ Improves model performance
- ✓ Handles missing or noisy data
- ✓ Makes data consistent
- ✓ Helps in faster training
- ✓ Better generalization

"Good data leads to Good Models."



Handling Missing Values

- **Remove:** Delete rows or columns with missing values.
- **Mean/Median/Mode Imputation:** Fill missing numerical values with mean/median and categorical with mode.
- **Forward/Backward Fill:** Use previous or next value (useful in time series).

Encoding Categorical Data

- Label Encoding (for Ordinal / Nominal)
- One-Hot Encoding (for Nominal)

Color	Encoded
Red	0
Green	1
Blue	2
Yellow	3

Red	Green	Blue	Yellow
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

Feature Scaling (Normalization)

① Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \rightarrow \text{Scaled between 0 and 1}$$

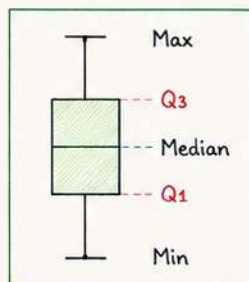
② Standardization (Z-score)

$$z = \frac{x - \mu}{\sigma} \rightarrow \text{Mean} = 0, \text{Std Dev} = 1$$

Outlier Detection

- Using IQR Method

- ① Q_1 = 25th percentile
- ② Q_3 = 75th percentile
- ③ $IQR = Q_3 - Q_1$
- ④ Outlier if:
 $x < Q_1 - 1.5 IQR$ or $x > Q_3 + 1.5 IQR$



Train - Test Split

- Split data into training and testing sets.

Training Set (70% - 80%)	Testing Set (20% - 30%)
-----------------------------	----------------------------

"Train the model on training data and evaluate its performance on unseen testing data. ★"

★ Preprocessing is the foundation of Machine Learning. Clean data = Better Models! ♥



3. HYPOTHESIS, COST FUNCTION & GRADIENT DESCENT



Hypothesis

A hypothesis is a function that maps input features (X) to output ($\hat{y}$).

Linear Regression Hypothesis

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

↑ ↑ ↑
 Output (Prediction) Parameters (To be learned) Input Features

θ_0 is bias (intercept) term.

Gradient Descent

An optimization algorithm used to minimize the cost function by updating parameters iteratively.

Update Rule (Simultaneous)

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

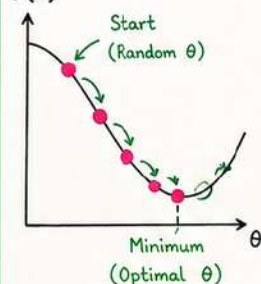
(for $j = 0, 1, 2, \dots, n$)

α = Learning rate (step size)

$\frac{\partial J(\theta)}{\partial \theta_j}$ = Partial derivative of cost w.r.t. θ_j

How Gradient Descent Works?

$J(\theta)$



1. Start with random θ
2. Calculate slope (gradient)
3. Update θ in the opposite direction of gradient
4. Repeat until convergence

Cost Function

Cost function measures how far our predictions are from actual values.

Mean Squared Error (MSE)

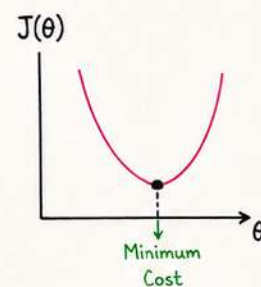
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

m = number of training examples

$h_{\theta}(x^{(i)})$ = predicted value

$y^{(i)}$ = actual value

$J(\theta)$ = cost (error)



Goal: Minimize $J(\theta)$

Normal Equation (Closed Form Solution)

For Linear Regression, parameters can also be computed directly.

$$\theta = (X^T X)^{-1} X^T y$$

X = Feature matrix ($m \times (n+1)$)

y = Output vector ($m \times 1$)

θ = Parameter vector ($(n+1) \times 1$)



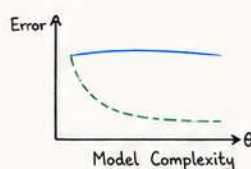
Gradient Descent works well for large datasets.

Normal Equation is faster for small datasets. 😊

Bias - Variance Tradeoff

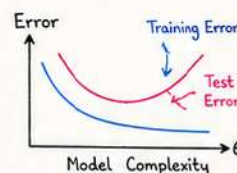
High Bias

- Model is too simple
- Underfitting
- High error on both training & test data



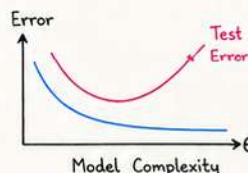
Optimal Balance

- Good fit
- Good generalization
- Low error on test data



High Variance

- Model is too complex
- Overfitting
- Low training error but high test error



The goal of ML is to find patterns that generalize well to unseen data! ❤️



4. CLASSIFICATION ALGORITHMS



Classification is a type of **Supervised Learning** where the model learns to predict a **discrete class label**.

Common Algorithms

- ① **Logistic Regression** – Uses sigmoid function to predict probability of classes.
- ② **K-Nearest Neighbors (KNN)** – Classifies based on majority vote of 'K' nearest neighbors.
- ③ **Decision Tree** – Splits data into branches based on feature conditions.
- ④ **Random Forest** – Ensemble of decision trees using bagging and feature randomness.
- ⑤ **Support Vector Machine (SVM)** – Finds the best hyperplane to separate classes.

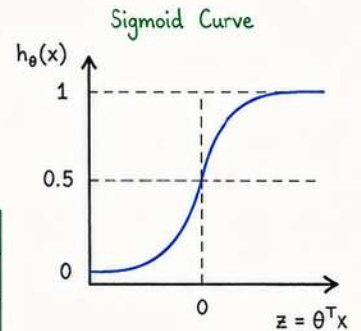
1. Logistic Regression

Sigmoid (Logistic) Function

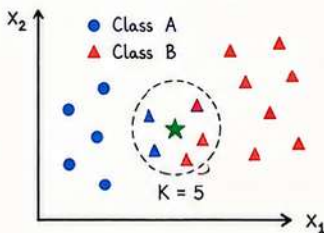
$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Output: Probability between 0 and 1

Probability	Predicted Class
$h_{\theta}(x) \geq 0.5$	1 (Positive)
$h_{\theta}(x) < 0.5$	0 (Negative)



2. K-Nearest Neighbors (KNN)



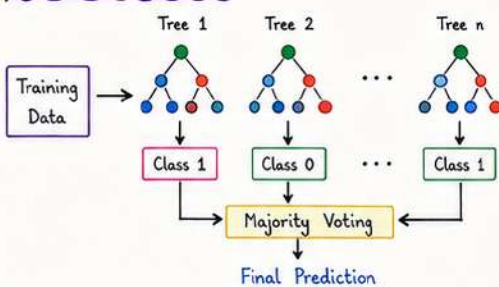
- Choose value of K.
- Calculate distance from the new point to all training points.
- Select K nearest neighbors.
- Assign the class by majority vote.

3. Decision Tree



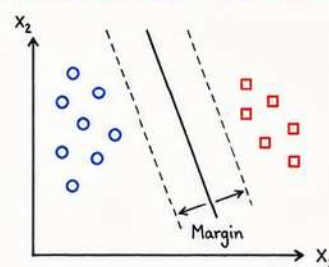
- * Easy to interpret
- * Handles both numerical & categorical data
- * Prone to overfitting (high variance)

4. Random Forest



- Builds many decision trees on random subsets.
- Each tree gives a prediction.
- Final output is the majority vote (classification).
- Reduces overfitting and improves accuracy.

5. Support Vector Machine (SVM)



- Finds the hyperplane that maximizes the margin between classes.
- Works well in high dimensional spaces.
- Can use different kernels (Linear, RBF, Polynomial).

Evaluation Metrics for Classification

Actual \ Predicted	Positive	Negative
Positive	TP (True Positive)	FN (False Negative)
Negative	FP (False Positive)	TN (True Negative)

• Accuracy = $\frac{TP + TN}{TP + TN + FP + FN}$

• Precision = $\frac{TP}{TP + FP}$

• Recall (Sensitivity) = $\frac{TP}{TP + FN}$

• F1-Score = $\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Choose the right algorithm based on data, problem type and evaluation metrics! 💡❤️



5. CLUSTERING, DIMENSIONALITY REDUCTION & OVERFITTING



1. Clustering (Unsupervised Learning)

Clustering groups similar data points together without using labels.

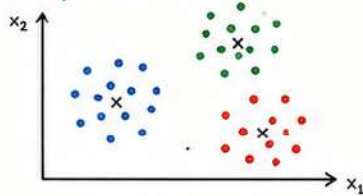
a) K-Means Clustering

- Choose K (number of clusters)
- Initialize K centroids randomly
- Assign each point to nearest centroid
- Update centroids (mean of points in cluster)
- Repeat steps 3-4 until convergence

Distance (Euclidean)

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Example (K = 3)



Applications

- Customer Segmentation
- Image Compression
- Document Clustering
- Anomaly Detection

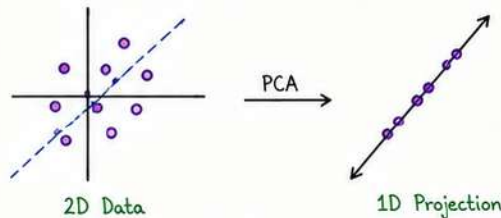
2. Dimensionality Reduction (PCA)

PCA (Principal Component Analysis) reduces the number of features while preserving maximum variance.

Steps:

- ① Standardize the data
- ② Compute covariance matrix
- ③ Compute eigenvalues and eigenvectors
- ④ Sort eigenvectors by decreasing eigenvalues
- ⑤ Choose top 'k' eigenvectors
- ⑥ Project data onto new k-dimensional space

Illustration (2D to 1D)



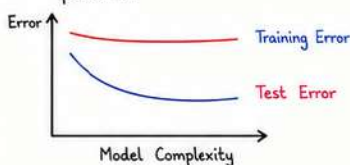
Benefits

- ✓ Reduces computational cost
- ✓ Removes redundancy
- ✓ Helps visualization

3. Overfitting vs Underfitting

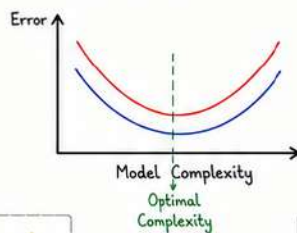
Underfitting

Model is too simple to capture the underlying pattern.



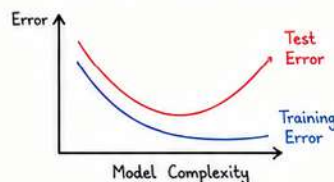
Good Fit

Model generalizes well to unseen data.



Overfitting

Model is too complex and fits noise in training data.



How to Prevent Overfitting?

- ✓ Get more training data
- ✓ Simplify the model
- ✓ Use Regularization (L1, L2)
- ✓ Apply Dropout (in Neural Networks)
- ✓ Use Cross-Validation
- ✓ Early Stopping



Regularization

Adds a penalty for large weights to reduce model complexity.

Ridge (L2) Regularization

$$J(w) = \text{Loss}(w) + \lambda \sum_{i=1}^n w_i^2$$

Lasso (L1) Regularization

$$J(w) = \text{Loss}(w) + \lambda \sum_{i=1}^n |w_i|$$

λ (lambda):

- Controls the strength of regularization
- Higher $\lambda \rightarrow$ Simpler model
- $\lambda = 0 \rightarrow$ No regularization

Summary

- Clustering $\rightarrow$ Groups similar data without labels.
- PCA $\rightarrow$ Reduces features while keeping important info.
- Overfitting $\rightarrow$ Too complex model, high test error.
- Regularization $\rightarrow$ Simplifies model and improves generalization.

Remember!

The goal of ML is not just to fit the training data, but to **generalize** well to unseen data.



Better Data $\rightarrow$ Better Features $\rightarrow$ Better Models $\rightarrow$ Better Decisions! 😊



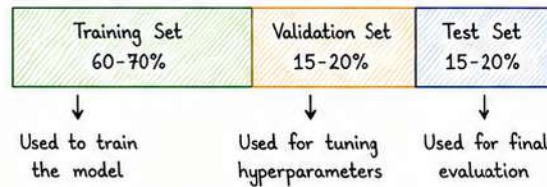
6. MODEL EVALUATION & HYPERPARAMETER TUNING



Why Evaluate Models?

- To ensure the model performs well on unseen data.
- To compare different models fairly.
- To detect overfitting or underfitting.

Train, Validation & Test Split



Key Points

- ★ Never evaluate on training data.
- ★ Use validation set for model selection and tuning.
- ★ Test set is used only once at the end.

1. Evaluation Metrics

For Classification

Metric	Formula	Best Value	Use When
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	1 (100%)	Classes are balanced
Precision	$\frac{TP}{TP + FP}$	1 (100%)	Minimize False Positives
Recall	$\frac{TP}{TP + FN}$	1 (100%)	Minimize False Negatives
F1-Score	$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	1 (100%)	Imbalanced classes

For Regression

Metric	Formula	Best Value	Use When
MAE	$\frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	0	Outliers present
MSE	$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	0	Penalize large errors
RMSE	$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$	0	Same unit as target
R ² Score	$1 - \frac{SS_{res}}{SS_{tot}}$	1	Explains variance

2. Confusion Matrix (Classification)

		Predicted	
		Positive	Negative
Actual	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

- TP: Model predicted Positive and it is Positive.
- TN: Model predicted Negative and it is Negative.
- FP: Model predicted Positive but it is Negative.
- FN: Model predicted Negative but it is Positive.

3. Hyperparameter Tuning

Why?

To find the best set of hyperparameters that gives the best performance.

Common Methods

- Grid Search: Try all combinations from a predefined grid.
- Random Search: Try random combinations.
- Bayesian Optimization: Smart search using past results.

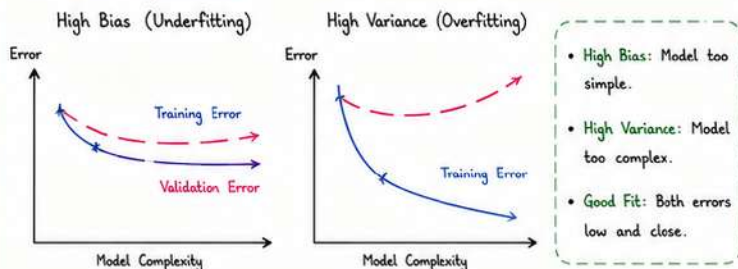
Grid Search Example

Learning Rate	Max Depth
0.01	3
0.01	5
0.01	7
0.1	3
0.1	5
0.1	7
...	...

Steps

- ① Define parameter grid
- ② Train model for each combination
- ③ Evaluate on validation set
- ④ Choose the best combination
- ⑤ Evaluate on test set

4. Learning Curves



5. Cross-Validation

When data is limited, use k-Fold Cross-Validation.

	Fold 1	1	2	3	4	4	5	k
Iteration 1	Validate	Train	Train	Train	Train	Train	Train	
Iteration 2	Train	Validate	Train	Train	Train	Train	Train	
Iteration 3	Train	Train	Validate	Train	Train	Train	Train	
...	...	...	...	...	...	...	...	
Iteration k	Train	Train	Train	Train	Train	Train	Validate	

- Each fold is used once for validation.
- Average the results across all folds.



Good evaluation + Proper tuning = Better Models + Better Decisions 😊

